

RAPORT UTILIZARE AI

Utilizarea Instrumentelor de Inteligență Artificială în Dezvoltare

PENTRU ALaCarteOS

1. Introducere

Acest raport documentează modul în care instrumentele de **Inteligență Artificială Generativă** (LLM — Large Language Models) au fost utilizate ca asistenți de programare (AI Pair-Programmers) pe parcursul ciclului de viață al dezvoltării proiectului web **ALaCarteOS**. Scopul utilizării acestor tehnologii nu a fost înlocuirea deciziei umane, ci augmentarea productivității, accelerarea procesului de scriere a codului (boilerplate), depanarea rapidă a erorilor și identificarea celor mai bune practici de arhitectură software.

2. Instrumente AI Utilizate

În procesul de dezvoltare au fost utilizate instrumente de asistență bazate pe modele de limbaj avansate (ex. Google Gemini / ChatGPT / Asistenți integrați în IDE). Aceste instrumente au fost folosite interactiv, prin tehnici de *Prompt Engineering*, solicitând generarea, explicarea sau corectarea unor secțiuni specifice de cod pe baza contextului furnizat de programator.

3. Integrarea AI în Ciclul de Viață al Dezvoltării Software (SDLC)

Asistența AI a fost integrată în patru faze majore ale dezvoltării proiectului:

3.1 Planificare și Arhitectură

În faza de început, instrumentele AI au fost folosite pentru brainstorming și structurare. S-au obținut recomandări privind:

- **Organizarea optimă** a structurii de directoare (ex: separarea fișierelor statice, template-urilor și modulelor logice).
- **Proiectarea modelelor relaționale** în Django (models.py), stabilind corect legăturile de tip One-to-Many și Many-to-Many între entități (Comenzi, Produse, Ingrediente, Mese).
- **Securizarea mediului** de dezvoltare prin implementarea variabilelor de mediu (.env) pentru a proteja cheile API și credențialele bazei de date.

3.2 Depanare (Debugging) și Rezolvarea Erorilor

Unul dintre cele mai mari avantaje ale utilizării AI a fost faza de troubleshooting. AI-ul a acționat ca un expert tehnic capabil să analizeze traceback-uri (loguri de eroare) complexe și să ofere soluții precise. Exemple concrete din proiect includ:

- **Erori de Securitate CSRF (403 Forbidden):** AI-ul a identificat rapid lipsa decoratorului `@csrf_exempt` pe rutele API și a explicat necesitatea curățării sesiunilor la deconectarea utilizatorilor, prevenind erorile de token.
- **Gestionarea Conexiunilor la Baza de Date:** La întâmpinarea erorii *FATAL: max_clients_reached* de la Supabase, AI-ul a diagnosticat imediat cauza și a sugerat modificarea parametrului `conn_max_age=0` în `settings.py`.
- **Migrarea SDK-urilor Depreciate:** Când Google a renunțat la vechiul pachet, AI-ul a asistat la rescrierea și migrarea întregului cod backend către noul pachet `google.genai`.

3.3 Refactorizare și Optimizare (Best Practices)

AI-ul a propus soluții arhitecturale pentru eficientizarea aplicației:

- **Implementarea Caching-ului:** Pentru a preveni epuizarea limitelor API, AI-ul a sugerat salvarea datelor în memoria RAM (`django.core.cache`) timp de 15 minute, reducând drastic numărul de interogări externe.
- **Sisteme de Fallback și Modul Demo:** AI-ul a ghidat implementarea unui sistem robust de erori și crearea variabilei `DEMO_MODE`, asigurând un mediu de prezentare stabil.

4. Limite și Necesitatea Supravegherii Umane

Deși productivitatea a crescut semnificativ, utilizarea AI-ului a subliniat importanța expertizei umane:

- **„Halucinații” de Cod:** Modelele AI au sugerat ocazional nume de module care nu mai existau sau funcții cu parametri incorecți. A fost necesară verificarea constantă a documentației oficiale.
- **Securitate și Logică de Business:** Deciziile de arhitectură critică (ex. logica din Modulul 86 care dezactivează automat produsele) au fost delegate programării algoritmice clasice, pentru a garanta determinismul și precizia matematică.

5. Concluzie

Utilizarea instrumentelor bazate pe inteligență artificială în dezvoltarea ALaCarteOS s-a dovedit a fi un catalizator esențial. Acestea au acționat ca un mentor tehnic disponibil 24/7, transformând orele de citire a documentațiilor și depanare în intervenții de câteva minute. Succesul integrării a depins de capacitatea dezvoltatorului uman de a formula cerințe precise, de a valida soluțiile propuse și de a asambla modulele generate într-o arhitectură software sigură, scalabilă și coerentă.